

Advanced Data Management

First-Order Logic and Databases (Relational Calculus)

Domenico Fabio Savo

*Corso di laurea magistrale
INGEGNERIA INFORMATICA
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione
University of Bergamo*

Outline of the course

1. Introduction to propositional logic
 2. Introduction to First-order Logic
 - 3. Relational Calculus**
 - 4. Information Integration Systems (IIS)**
 5. Logical formalization of IISs
 6. Mapping between **Global Schema** e **Data Sources**
 7. Incomplete information databases
 8. Query answering over IISs
 - 9. Ontology-Based Data Management (OBDM)**
 10. Description Logic Ontologies
 11. Query answering in ontologies
 12. Query answering in OBDM
-

The Relational Model of Data

Database Models

- The goal of Information Integration is to provide a uniform view of data coming from multiple and heterogenous sources.
 - SQL Technologies, Spreadsheets, XML\JSON, CSV, ...
- To abstract from the details of each single source containing the information, we often assume that data are contained in a **database**.
 - We can safely assume that the wrappers present the data this way.
- **What is a database?** To develop a formal theory of information integration, we need to formalize the notion of database.

The Relational Model of Data

Proposed by E. F. Codd in 1970 for the **physical independence** of data. Indeed, the way data is used at the logical level is independent from their physical storage.

Made available in commercial **DBMSs** in 1981 (it is not easy to implement data independence efficiently and reliably!)

It is based on (a variant of) the mathematical notion of **relation**

Relations admit a very natural and intuitive representation as **tables**.

Mathematical Relations

- Consider n sets $D_1, D_2, \dots, D_n$, not necessarily distinct.
- The **cartesian product** $D_1 \times D_2 \times \dots \times D_n$, is the set of **all** n -tuples $(d_1, d_2, \dots, d_n)$ such that $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$.
- A (mathematical) **relation** over $D_1, D_2, \dots, D_n$ is a **subset of their cartesian product** $D_1 \times D_2 \times \dots \times D_n$.
- When **R** is a mathematical relation over the sets $D_1, D_2, \dots, D_n$, such sets are called the **domains** or the relation **R**. A relation over n domains has **arity** n .
- The number of n -tuples is the **cardinality** of the relation.

Mathematical Relations: Example

- $D_1 = \{\text{Pizza, Ramen}\}$ (delicious food)
- $D_2 = \{\text{Italy, Japan}\}$ (countries)

Cartesian product $D_1 \times D_2 \longrightarrow$

Pizza	Italy
Pazza	Japan
Ramen	Italy
Ramen	Japan

A relation: **Origin** $\subseteq D_1 \times D_2 \longrightarrow$

Pizza	Italy
Ramen	Japan

Relational Databases

In what follows we assume to have the following pairwise distinct countably infinite sets:

- **Cons** of constants
- **Rels** of relational predicates
- **Vars** of variables

and the set:

- **Terms = Vars $\cup$ Cons**
-

Relational Databases: Schema

Relation schema:

- A relational predicate $R \in \mathbf{Rels}$ with a set of attributes $X = \{A_1, \dots, A_n\}$ over the domains $D_1, \dots, D_n$.
- It is denoted by $R(A_1:D_1, \dots, A_n:D_n)$ or simply $R(A_1, \dots, A_n)$.
- The integer n is called the **arity** of R and is denoted by $ar(R)$. We write R_n if R has $ar(R) = n$.

Exams

Student

Grade

Course



Exams(Student, Grade, Course)

Relational Database schema:

a set of **relation schemas** all with different names $S = \{R_1(X_1), \dots, R_n(X_n)\}$

Relational Databases: Instances

Relation:

- A **relation** over a schema $R(X)$ such that $ar(R) = n$ is a set r of n -tuples on X .
- It represents an **instance** of a relation schema

<u>Exams</u>	Student	Grade	Course
	3456	30	04
	3254	24	05

Instance of the relation Exams

Database:

- A **database** D over a schema $S = \{R_1(X_1), \dots, R_n(X_n)\}$ is a set of relations $r = \{r_1, \dots, r_n\}$ (with r_i relation on R_i). We denote with $D(R_i)$ the relation r_i .
- Given a database D we denote by $dom(D)$ the domain of D , i.e., the set of constants occurring in D . We always have that $dom(D) \subseteq Cons$.
- Sometimes in this course we will see a database as a set of facts of the form $R(v_1, v_2, \dots, v_n)$ where $(v_1, v_2, \dots, v_n)$ is a n -tuple over the relation schema R_n

Example of a database

Products

Code	Color
P001	Red
P002	Green
P003	Blue

Customers

Code	Name
C001	Eryn
C002	Dylan
C003	Cora

Orders

Code	Customer	Product	Quantity
O001	C001	P001	4
O002	C002	P002	2
O003	C001	P001	3

Summary of Definitions

Relation name: a symbol representing the relation and its arity.

$$R_{/k}$$

Database schema: non-empty set of relation schemas with different names.

$$S = \{ R_1, \dots, R_n \}$$

Tuple: An ordered sequence of domain elements.

$$t = \langle v_1, \dots, v_n \rangle$$

Relation (instance): of $R_{/k}$ is a set of n-tuples.

$$\{t_1, \dots, t_m\}$$

Database (instance): over S . A function D from relation names to instances.

$$D(R_1) = \dots, \dots, D(R_n) =$$

Integrity Constraints

Not all database instances represent information that is consistent with our knowledge of the domain of interest

To define valid instances we can use **integrity constraints**, i.e., conditions written in a formal language that all valid database instances must respect

The relational model allows two different forms of constraints

- **Intra-relational**: condition specified within a single relation
 - **Inter-relational**: condition specified across different relations
-

Integrity Constraints: motivations

- Useful to describe the domain in **greater detail**
- A contribution to “**data quality**”
- An element in the **design process**
- Used by the system in choosing the strategy for **query** processing

Note: not all the properties can be described by means of integrity constraints

Key Constraints

- **Intuition:** A **key constraint** is a statement that specifies that a (non-empty) set of tuple components must be unique in a relation.
- **Syntax:** Key: $R[A_1, \dots, A_k]$ where $A_1, \dots, A_k$ are attributes of the relation R .
- **Semantics:** Key: $R[A_1, \dots, A_k]$ is satisfied on a database instance D if there are no two distinct tuples in $D(R)$ that agree on attributes $A_1, \dots, A_k$

Key Constraints: Example

<u>Student</u>	Id	Surname
	1	Rossi
	2	Rossi
	3	Verdi

The above database satisfies the key constraint:

key: Student[Id]

While it does not satisfy the key constraint:

key: Student[Surname]

Inclusion Constraints

- **Intuition:** An **inclusion constraint** imposes to the values on a (non-empty) sequence X of attributes of a relation R to occur as values in a specific sequence Y of attributes of a different relation S .
- **Syntax:** $R(X) \subseteq S(Y)$
Where X and Y are non-empty sequences of both n attributes of R and S , respectively.
- **Semantics:** $R(\{i_1, i_2, \dots, i_n\}) \subseteq S(\{j_1, j_2, \dots, j_n\})$ is satisfied in an instance D if for each tuple t_1 in $D(R)$ there exists a tuple t_2 in $D(S)$ such that $t_1[i_k] = t_2[j_k]$, for each k in $\{1, \dots, n\}$

Inclusion Constraints: Example

Student

Id	Surname
1	Rossi
2	Rossi
3	Verdi

Grades

Student	Exam	Exam
1	Math101	A
2	CS101	B
2	CS102	C

The above database satisfies the inclusion constraint:

$$\text{Grades}[\text{Student}] \subseteq \text{Student}[\text{Id}]$$

While it does not satisfy the inclusion constraint:

$$\text{Student}[\text{Id}] \subseteq \text{Grades}[\text{Student}]$$

Database Queries

- **What is a database query?**
 - **Intuitively:** a way to extract information from a database.
 - Expressed in some language (SQL?)
 - Each expression has an associated meaning, i.e., the result of evaluating the query.
 - How do we formally define database queries?
-

Database Queries – Formal Definition

- Assume sets of database $\mathcal{D}$ and a set of relations $\mathcal{R}$.
- A **database query** is a function $q: \mathcal{D} \rightarrow \mathcal{R}$.
 - Intuitively, given a $D \in \mathcal{D}$, $q(D) \in \mathcal{R}$ is the relation corresponding to the set of answers for q over D .
- This definition is general enough to encompass all possible queries.

Query Languages

We need to consider queries that we **can actually handle**.

- We need to be able to write them down in a compact way.
- We need to clearly understand the complexity of evaluating queries.

A **query language** is a set of “syntactic” objects usually called *query expressions* or simply *queries*.

Query expressions have an attached meaning.

- Procedurally (relational algebra)
- Declaratively (relational calculus)

Logic And Databases

Logical Formulae and Logical Theories

- We can use logical formulae in two essential ways
 1. We can evaluate the value of a formula over an interpretation
 - Given an assignment, a formula is either true or false in an interpretation.
 2. We can use formulae to describe a set of “correct” interpretations.
 - All those that satisfy the formulae (independently from the chosen assignment).
 - In this case, we talk about logical theories.
 - Interpretations satisfying a theory represent valid possible worlds.
- Logical theories and logical formulae are a staple of databases.
 - Can you guess why?

Databases and First-Order Logic

We can see a database instance D as a (finite) interpretation I .

- Our **domain** is the set $\text{dom}(D)$ of attribute values
- Our **alphabet** is formed by:
 - the set **Rel**s of relational predicates R/n
 - the set **Cons** of constants (we have a function c with arity 0 (constant) for each element c in dom).

We use **databases as interpretations**. Hence, given a database D over a schema $S = \{R_1, \dots, R_n\}$, we implicitly see D as the interpretation $I = (\text{dom}(D), \cdot^I)$, where:

- $R^I = D(R)$ for each $R \in S$ and $R^I = \emptyset$ for each $R \notin S$
- $c^I = c$ for each $c \in \text{Cons}$ (each constant is interpreted into itself)

Exercise

Write the interpretation that represents the following database.

Products

Code	Color
P001	Red
P002	Green
P003	Blue

Customers

Code	Name
C001	Eryn
C002	Dylan
C003	Cora

Orders

Code	Customer	Product	Quantity
O001	C001	P001	4
O002	C002	P002	2
O003	C001	P001	3

Exercise 1: Solution

- **Rels** = $\{Products, Customers, Orders \dots\}$
- **Cons** = $\{P001, P002, P003, Red, Green, Blue, C001, C002, C003, Eryn, Dylan, Cora, O001, O002, O003, 4, 3 \dots\}$
- $\Delta^I = \{P001, P002, P003, Red, Green, Blue, C001, C002, C003, Eryn, Dylan, Cora, O001, O002, O003, 4, 3 \}$
- $Products^I = \{(P001, Red), (P002, Green), (P003, Blue)\}$
- $Customers^I = \{(C001, Eryn), (C002, Dylan), (C003, Cora)\}$
- $Orders^I = \{(O001, C001, P001, 4), (O002, C002, P002, 4), (O001, C001, P001, 3)\}$
- $c^I = c$ for each $c \in \mathbf{Cons}$

FOL As a Query Language

Open and Closed Formulae

Assume a FOL formula φ .

- A variable x occurring in φ is **bound** if it occurs in the scope of an existential quantifier. A variable is **free** if it is not bound.
- An **open** formula is a FOL formula **with** free variables.
- A **closed** formula (sentence) is a FOL formula **without** free variables.

Closed formulae do not need assignments to be evaluated.

- For such formulae, we write $I \models \varphi$ instead of $I, \alpha \models \varphi$ (when this causes no confusion)

Open formulae require assignments to be evaluated (because of the free variables), so they are the perfect candidate for a query language!

Exercise

Which of the following is an open formula?

1. $\exists x. \forall y. (P(x, y) \wedge R(y, z))$

2. $\exists x. \forall y. (P(x, y) \wedge \exists z. (R(y, z)))$

3. $\exists x. \forall y. \exists z. (P(x, y) \wedge R(y, z))$

4. $\exists x. ((\forall y. \exists z. P(x, y)) \wedge R(x, z))$

Exercise: Solution

Which of the following is an open formula?

1. $\exists x. \forall y. (P(x, y) \wedge R(y, \mathbf{z}))$

- **Yes**

2. $\exists x. \forall y. (P(x, y) \wedge \exists z. (R(y, z)))$

- **No**

3. $\exists x. \forall y. \exists z. (P(x, y) \wedge R(y, z))$

- **No**

4. $\exists x. ((\forall y. \exists z. P(x, y)) \wedge R(x, \mathbf{z}))$

- **Yes**

FOL Queries

- Given a formula φ , we write $\varphi(x_1, \dots, x_n)$ to denote the fact that $x_1, \dots, x_n$ are the free variables of φ .
 - Observe that this notation implicitly give us an ordering over $x_1, \dots, x_n$
 - We assume that no function of arity greater than 0 occurs in φ .
- **Definition:** A FOL Query q is an expression of the form
$$q = \{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$$
- Given a database schema $\mathbf{S}$, we say that a FOL query $\mathbf{q}$ as above is a query **over** $\mathbf{S}$ if φ uses only relational predicates in $\mathbf{S}$.

FOL Queries: Answers

- Assume a FOL Query $q = \{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$ and an interpretation I
- Given $\varphi(x_1, \dots, x_n)$, we use $\langle c_1, \dots, c_n \rangle$ to denote an assignment α such that $\alpha(x_i) = c_i$, for $i = 1, \dots, n$
- **Definition:** The **answers for q over I** (written $q(I)$) are defined as follows

$$q(I) = \{(c_1, \dots, c_n) \mid I, \langle c_1, \dots, c_n \rangle \models \varphi(x_1, \dots, x_n)\}$$

FOL Boolean Queries

- A **Boolean FOL query** q is a FOL query of the form $\{ () \mid \varphi \}$.
 - In other words, a Boolean FOL query is defined using a **closed** formula.
- The **answers for q over I** (written q^I) are defined as follows
$$\{ () \mid I, \langle \rangle \models \varphi(x_1, \dots, x_n) \}$$
- **Observe:** if q is Boolean, then $q(I) = \emptyset$ or $q(I) = \{ () \}$, i.e., $q(I)$ is either empty or contains only the empty tuple.
 - We understand the former as **false** and the latter as **true**.

Exercise

Consider the following database D and the FOL query:

$$q = \{(x) \mid \exists y. \exists z. (Customers(y, x) \wedge Orders(z, y, P002))\}$$

Products

Code	Color
P001	Red
P002	Green
P003	Blue

Customers

Code	Name
C001	Eryn
C002	Dylan
C003	Cora

Orders

Code	Customer	Product
O001	C001	P001
O002	C002	P002
O003	C001	P001

- Write an equivalent SQL query.
- What are the answers of such query?

Exercise: Solution

- $q = \{(x) \mid \exists y. \exists z. (Customers(y, x) \wedge Orders(z, y, P002))\}$
- ```
SELECT Customers.name
FROM Customers, Orders
WHERE Customers.Code = Orders.Customer and
Orders.Product = "P002"
```
- $q(D) = \{(Dylan)\}$

| Name  |
|-------|
| Dylan |

# Complexity of FOL Queries

---

# Computational complexity – Recap

Computational complexity theory aims to classify computational problems with respect to the resources (space and time) they use.

We are interested in class of **decision problems**, i.e., problems that provide a “**yes**” or a “**no**” answer based on their input.

**Example:** given a string  $s$  and a language  $L$  decide if  $s$  belongs to  $L$

# Evaluating FOL Queries

Evaluating FOL queries over finite interpretations is **decidable**.

- **Query Answering Problem** (Recognition): Given a finite interpretation  $I$ , FOL formula  $\varphi(x_1, \dots, x_n)$ , and a tuple  $\langle c_1, \dots, c_n \rangle$ , check whether  $I, \langle c_1, \dots, c_n \rangle \models \varphi(x_1, \dots, x_n)$ .

**What is the complexity of Query Answering?**

# Evaluating FOL Queries

- Query Answering Problem (Recognition): Given a finite interpretation  $I$ , FOL formula  $\varphi(x_1, \dots, x_n)$ , and a tuple  $\langle c_1, \dots, c_n \rangle$ , check whether  $I, \langle c_1, \dots, c_n \rangle \models \varphi(x_1, \dots, x_n)$ .
- We can define an algorithm to solve the Query Answering Problem
  - Runs in  $O(|I| + |\varphi|^{|\varphi|})$  time.
  - Requires  $O(|I| \cdot |\varphi|)$  space.
- Therefore, the algorithm is exponential. However:
  - Exponential in the **size of the query** (usually small)
  - Polynomial in the **size of the database** (usually large)

# Different Complexities

- **Data Complexity:** The complexity of the problem for a **fixed query**.
  - **Query Complexity:** The complexity of the problem for a **fixed interpretation (so fixed database)**.
  - **Combined Complexity:** The complexity of the **whole problem (query + interpretation)**
-



# Complexities of FOL Query Answering

- **Thm:** Query Answering is PSPACE-complete in Combined Complexity.
- **Corollary:** Query Answering is in EXPTIME in Combined Complexity.
- **Thm:** Query Answering is in LOGSPACE in Data Complexity.
- **Corollary:** Query Answering is in PTIME in Data Complexity.
- **Thm:** Query Answering is PSPACE-complete in Query Complexity.
- **Corollary:** Query Answering is in EXPTIME in Query Complexity.

# Conjunctive Query

- A FOL formula  $\varphi(x_1, \dots, x_n)$  is an existential conjunction if it is of the form

$$\exists y_1, \dots, \exists y_n P_1(\bar{z}_1) \wedge \dots \wedge P_k(\bar{z}_k)$$

- A FOL query  $\{(x_1, \dots, x_n) | \varphi(x_1, \dots, x_n)\}$  is a **Conjunctive Query** if  $\varphi(x_1, \dots, x_n)$  is an existential conjunction.

# Union of Conjunctive Queries

- A formula  $\varphi(x_1, \dots, x_n)$  is Union of Existential Conjunctions if it is of the form

$$\bigvee_i \psi_i(x_1, \dots, x_n)$$

where each  $\psi_i$  is an existential conjunction. **Observe:** same free variables.

- A FOL query  $\{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$  is a **Union of Conjunctive Queries** if  $\varphi(x_1, \dots, x_n)$  is a Union of Existential Conjunctions.

# Complexities of UCQ Answering

- **Thm:** Query Answering is NP-complete in Combined Complexity for UCQs
  - **Thm:** Query Answering is in LOGSPACE in Data Complexity.
  - **Corollary:** Query Answering is in PTIME in Data Complexity.
  - **Thm:** Query Answering is NP-complete in Query Complexity for UCQs.
-

# FOL Queries Exercises

---

# Exercise

Consider once again the following database

**Products**

| Code | Color |
|------|-------|
| P001 | Red   |
| P002 | Green |
| P003 | Blue  |

**Customers**

| Code | Name  |
|------|-------|
| C001 | Eryn  |
| C002 | Dylan |
| C003 | Cora  |

**Orders**

| Code | Customer | Product |
|------|----------|---------|
| O001 | C001     | P001    |
| O002 | C002     | P002    |
| O003 | C001     | P001    |

# Interpretation

- **Rels** =  $\{Products, Customers, Orders\}$
- **Cons** =  $\{P001, P002, P003, Red, Green, Blue, C001, C002, C003, Eryn, Dylan, Cora, O001, O002, O003\}$
- $\Delta^I = \{P001, P002, P003, Red, Green, Blue, C001, C002, C003, Eryn, Dylan, Cora, O001, O002, O003\}$
- $Products^I = \{(P001, Red), (P002, Green), (P003, Blue)\}$
- $Customers^I = \{(C001, Eryn), (C002, Dylan), (C003, Cora)\}$
- $Orders^I = \{(O001, C001, P001), (O002, C002, P002), (O001, C001, P001)\}$
- $c^I = c$  for each  $c \in \mathbf{Cons}$

## Example

Write a FOL query that returns the codes of the customers who ordered a red product.



## Example

Write a FOL query that returns the codes of the customers who ordered a red product.

$$\{(y) \mid \exists x. \exists z. \textit{Orders}(x, y, z) \wedge \textit{Products}(z, \text{"red"}) \}$$

## Exercise 3

Write a FOL query, a Relational algebra expression, and an SQL expression for the following query:

"Return the code of those customers who have ordered at least one item"

## Exercise 3 - Solution

Write a FOL query, a Relational algebra expression, and an SQL expression for the following query:

"Return the code of those customers who have ordered at least one item"

- $\{(y) \mid \exists x. \exists z. Orders(x, y, z) \}$
- $PROJ_2 (Orders)$
- **SELECT Customer FROM Orders**

## Exercise 4

Write a FOL query, a Relational algebra expression, and an SQL expression for the following query:

"Return the names of those customers who ordered product P001"

## Exercise 4 - Solution

Write a FOL query, a Relational algebra expression, and an SQL expression for the following query:

"Return the names of those customers who ordered product P001"

- $\{(x) \mid \exists y. \exists z. Orders(y, z, P001) \wedge Customers(z, x)\}$
- $PROJ_3(SEL_{1=2}(PROJ_2(SEL_3="P001"(Orders)) \times Customers))$
- **SELECT c.name FROM Orders o, Customers c  
WHERE o.Product = "P001" and o.customer = c.code**

## Exercise 5

Write a FOL query, a Relational algebra expression, and an SQL expression for the following query:

"Find the codes of those customers who ordered a red product"

## Exercise 5 - Solution

Write a FOL query, a Relational algebra expression, and an SQL expression for the following query:

"Find the codes of those customers who ordered a red product"

- $\{(y) \mid \exists x. \exists z. Orders(x, y, w) \wedge Products(w, Red)\}$
- $PROJ_2(SEL_{3=4 \wedge 5="Red"}(Orders \times Products))$
- ```
SELECT Customer FROM Orders o, Products p
WHERE o.Product = Products.id and
      p.color = "Red"
```

Exercise 6

Write a FOL query, and R.A. expression for the following query:

"Find the codes of customers who ordered a red and a green product"

Exercise 6 - Solution

Write a FOL query, and R.A. expression for the following query:

"Find the codes of customers who ordered a red and a green product"

- $\{(x) \mid \exists y. \exists z. \exists y'. \exists z'. (Orders(y, x, z) \wedge Products(z, Green)) \wedge (Orders(y', x, z') \wedge Products(z', Red))\}$
- $PROJ_2(SEL_{3=4}((SEL_{5="red"}(Orders \times Products))) \cap PROJ_2(SEL_{3=4}((SEL_{5="green"}(Orders \times Products))))$

Exercise 7

Write a FOL query, and a R.A. expression for the following query:

"Find the codes of customers who ordered a red or a green product"

Exercise 7 - Solution

Write a FOL query, and a R.A. expression for the following query:

"Find the codes of customers who ordered a red or a green product"

- $\{(x) \mid \exists y. \exists z. \exists y'. \exists z'. (Orders(y, x, z) \wedge Products(z, Green)) \vee (Orders(y', x, z') \wedge Products(z', Red))\}$
- $PROJ_2(SEL_{3=4}((SEL_{5="red"}(Orders \times Products))) \cup PROJ_2(SEL_{3=4}((SEL_{5="green"}(Orders \times Products))))$

Exercise 8

Write a FOL query, and a R.A. expression for the following query:

"Find the code of customers who have ordered the same product twice"

Exercise 8 - Solution

Write a FOL query, and a R.A. expression for the following query:

"Find the code of customers who have ordered the same product twice"

- $\{(y) \mid \exists x. \exists x'. \exists z. Orders(x, y, z) \wedge Orders(x', y, z) \wedge \neg(x = x')\}$
- $PROJ_2(SEL_{2=5}(SEL_{3=6}(SEL_{1 \neq 4}(Orders \times Orders))))$
- We can avoid the explicit use of $1 \neq 4$ using difference but the solution is much more involved.

Exercise 9

Write a FOL query, and a R.A. expression for the following query:

"Find the code of customers who have not ordered a red item"

Exercise 9 - Solution

Write a FOL query, and a R.A. expression for the following query:

"Find the code of customers who have not ordered a red item"

- **Solution**

- $\{(y) \mid \exists w. Customers(y, w) \wedge (\neg \exists x. \exists z. (Orders(x, y, z) \wedge Products(z, Red)))\}$
- $\{(y) \mid \exists w. Customers(y, w) \wedge (\forall x. \forall z. \neg(Orders(x, y, z) \wedge Products(z, Red)))\}$

- **Observe:** the following is wrong. Why?

- $\{(y) \mid \exists w. Customers(y, w) \wedge \exists x. \exists z. \neg(Orders(x, y, z) \wedge Products(z, "Red"))\}$

Exercise 10

Write a FOL query and a R.A. expression for the following query:

"Find the code of customers who have ordered all available products"

Exercise 10 - Solution

Write a FOL query and a R.A. expression for the following query:

"Find the code of customers who have ordered all available products"

-- We assume that all the available products are in Products

- $\{(x) \mid \exists n. Customers(x, n) \wedge \forall y. \forall z. Products(y, z) \rightarrow \exists w. Orders(w, x, y)\}$
- $PROJ_1(Customers) - PROJ_2(e)$
- $e = (PROJ_1(Products) \times PROJ_1(Customer)) - PROJ_{2,3}(Orders)$

Exercise 11

Write a FOL query, and a R.A. expression for the following query:

"Find the names of customers who have ordered all available green products"

Exercise 11 - Solution

Write a FOL query, and a R.A. expression for the following query:

"Find the names of customers who have ordered all available green products"

- $\{(x) | \exists z. Customers(z, x) \wedge \forall y. Products(y, Green) \rightarrow \exists w. Orders(w, x, y)\}$
- $PROJ_1(Customers) - PROJ_2(e)$
- $e = \left(PROJ_1(SEL_{2="green"}(Products)) \times PROJ_1(Customer) \right) - PROJ_{2,3}(Orders)$

FOL Integrity Constraints Exercises

FOL Constraints

- A set of first-order formulae can define integrity constraints.
 - A database satisfies the set of constraints if it is a model of the associated logical theory
 - Usually, constraints are expressed as **closed formulae**.
 - FOL can express standard constraints in the relational model.
-

FOL Constraints -- Examples

Assume relations $R_{/3}$ and $S_{/3}$. Write in FOL the following constraints.

- $R(1,2) \subseteq S(2,3)$
 - $R(1,2)$ is a key of $R_{/3}$
-

FOL Constraints -- Examples

Assume relations $R_{/3}$ and $S_{/3}$. Write in FOL the following constraints.

- $R(1,2) \subseteq S(2,3)$
 - $\forall x. \forall y. \forall z. R(x, y, z) \rightarrow \exists w. S(w, x, y)$
- $R(1,2)$ is a key of $R_{/3}$
 - $\forall x. \forall y. \forall z. \forall w. R(x, y, z) \wedge R(x, y, w) \rightarrow (z = w)$

Exercise 12

Consider once again the following database

Products

Code	Color
P001	Red
P002	Green
P003	Blue

Customers

Code	Name
C001	Eryn
C002	Dylan
C003	Cora

Orders

Code	Customer	Product
O001	C001	P001
O002	C002	P002
O003	C001	P001

Interpretation

- **Rels** = $\{Products, Customers, Orders\}$
- **Cons** = $\{P001, P002, P003, Red, Green, Blue, C001, C002, C003, Eryn, Dylan, Cora, O001, O002, O003\}$
- $\Delta^I = \{P001, P002, P003, Red, Green, Blue, C001, C002, C003, Eryn, Dylan, Cora, O001, O002, O003\}$
- $Products^I = \{(P001, Red), (P002, Green), (P003, Blue)\}$
- $Customers^I = \{(C001, Eryn), (C002, Dylan), (C003, Cora)\}$
- $Orders^I = \{(O001, C001, P001), (O002, C002, P002), (O001, C001, P001)\}$
- $c^I = c$ for each $c \in \mathbf{Cons}$

Exercise 12

Write in first-order logic all the integrity constraints you think are reasonable for the database above.

Exercise 12 - Solution

Foreign Keys

- $\forall x. \forall y. \forall z. Orders(x, y, z) \rightarrow \exists w Customers(y, w)$ (***Orders(2) $\subseteq$ Customers(1)***)
- $\forall x. \forall y. \forall z. Orders(x, y, z) \rightarrow \exists w Products(z, w)$ (***Orders(3) $\subseteq$ Products(1)***)

Key Constraints

- $\forall x. \forall y. \forall z. \forall y'. \forall z'. Orders(x, y, z) \wedge Orders(x, y', z') \rightarrow y = y' \wedge z = z'$
- $\forall x. \forall y. \forall y'. Customers(x, y) \wedge Customers(x, y') \rightarrow y = y'$
- $\forall x. \forall y. \forall y'. Products(x, y) \wedge Products(x, y') \rightarrow y = y'$

Other constraints

- $\forall x. \forall y. Customers(x, y) \rightarrow \exists z. \exists w Orders(z, x, w)$
- Every customer made at least one order

Credits

The presentations of this module of the course are based on the original slides of Prof. Giuseppe De Giacomo, Prof. Diego Calvanese, Prof. Marco Console, Prof. Maurizio Lenzerini, and Prof. Domenico Lembo.
